Análise de Eficácia de Testes com teste de Mutação

Disciplina: Teste de Software
Aluna: Brenda Evers
Matrícula: 854362
Repositório: https://github.com/eversbrehh/analise-e-refatoracao-de-mutantes-de-teste

1. Análise inicial

Ao executar *npm test -- --coverage*, a suíte original apresentou 98,65% de cobertura de linhas e 100% de cobertura de funções, mas apenas 58,82% de cobertura de ramos. A análise de mutação, executada com *npx stryker run*, gerou 213 mutantes e resultou em uma pontuação de mutação de 73,71%: 154 mutantes mortos, 3 encerrados por timeout, 44 sobreviventes e 12 sem cobertura.
A discrepância aponta que executar uma linha não está diretamente relacionado com verificar seu comportamento, uma vez que cada mutante vivo é uma alteração no código-fonte que sobrevive quando todos seus testes continuam passando. Os 56 mutantes não detectados indicam defeitos que a suíte original deixaria passar, apesar da cobertura de linhas próxima a 100%.
A *imagem_1 - Relatório StrykerJS* ilustra o relatório retornado pelo *StrykerJS* após sua primeira execução.

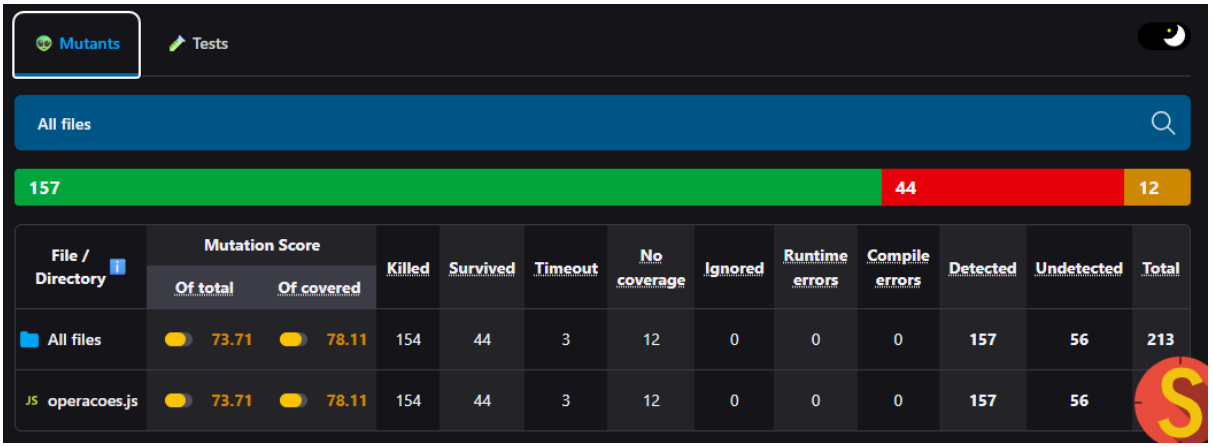


imagem_1 - Relatório StrykerJS

2. Análise de Mutantes Críticos

2. 1. Mutante 1 — isMaiorQue (linha 104, EqualityOperator)

O Stryker trocou *a > b* por *a >= b*. O único teste existente era *isMaiorQue(10, 5)*, que retorna *true* nas duas versões. Como nenhum teste usava valores iguais, o mutante sobreviveu no único caso em que os operadores diferem.

```
104   function isMaiorQue(a, b) { return a > b; }
```

imagem_2 - Trecho extraído de operacoes.js

2. 2. Mutante 2 — medianaArray (linha 109, MethodExpression)

O Stryker removeu a chamada a *.sort()*, fazendo a função calcular a mediana sobre o array na ordem recebida. O teste original usava *[1, 2, 3, 4, 5]*, que está ordenado, portanto a ordenação não fazia diferença no resultado.

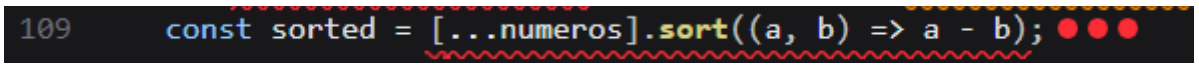


imagem_3 - Trecho extraído de operacoes.js

2. 3. Mutante 3 — celsiusParaFahrenheit (linha 93, ArithmeticOperator)

O Stryker trocou *celsius * 9* por *celsius / 9*. O teste original convertia 0 °C, e zero multiplicado ou dividido por qualquer valor continua zero: o resultado era 32 nas duas versões. A entrada escolhida anulava a operação que deveria ser verificada.

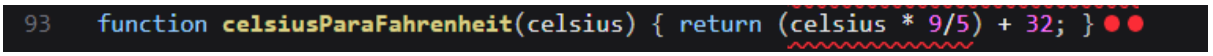


imagem_4 - Trecho extraído de operacoes.js

3. Solução Implementada

Foram adicionados 34 testes à suíte, mantendo os 50 originais. Para os mutantes analisados:

- **isMaiorQue**: *expect(isMaiorQue(5, 5)).toBe(false)*. Com >= o resultado passa a ser true e o teste falha. O mesmo foi feito para *isMenorQue*.
- **medianaArray**: e*xpect(medianaArray([5, 1, 3])).toBe(3)*. Sem a ordenação, a função devolve o elemento central da entrada (1). Também foi incluído um array de tamanho par, *[4, 1, 3, 2]*, cujo ramo não era executado por nenhum teste.
- **celsiusParaFahrenheit**: *expect(celsiusParaFahrenheit(100)).toBe(212)*. Com a divisão no lugar da multiplicação o resultado seria aproximadamente 34,2.

Os demais testes seguem os mesmos princípios: verificar o caso falso de cada predicado (isPar(3), isPrimo(4)), exercitar os valores de fronteira (raizQuadrada(0), fatorial(0), mediaArray([])), cobrir os ramos de erro conferindo a mensagem exata da exceção, e testar clamp fora do intervalo. São eficazes porque cada um usa uma entrada para a qual o código original e o mutado produzem resultados diferentes.

4. Resultados Finais

Após a inclusão dos novos testes, a pontuação de mutação passou de 73,71% para 97,65%: dos 213 mutantes, 205 foram mortos, 3 encerrados por timeout e 5 sobreviveram. Não houveram mutantes sem cobertura, e a cobertura de código chegou a 100% em linhas e ramos. A *tabela_1 - análise comparativa inicial e final* apresenta o resumo comparativo entre a baseline inicial e final, anterior e posterior as correções, respectivamente.

Métrica	Inicial	Final
Testes	50	84
Cobertura de linhas	98,65%	100%

Cobertura de ramos	58,82%	100%
Pontuação de mutação	73,71%	97,65%
Mutantes não detectados	56	5

tabela_1 - análise comparativa inicial e final

Os 5 sobreviventes são mutantes equivalentes, isto é, alterações que não mudam o comportamento observável do código e que, portanto, nenhum teste é capaz de matar:

- **fatorial, linha 19 (4 mutantes):** a condição *if (n === 0 || n === 1) return 1* é redundante. Sem ela, a variável resultado começa em 1 e o laço, que parte de i = 2, não executa para 0 nem para 1, de modo que o retorno continua sendo 1. Remover a condição, anular cada um dos seus lados ou trocar || por && produz o mesmo resultado para toda entrada.
- **produtoArray, linha 84 (1 mutante):** a condição *if (numeros.length === 0) return 1* também é redundante, pois *reduce* com valor inicial 1 devolve 1 para um array vazio.

Como 5 de 213 mutantes correspondem a 2,35%, o teto alcançável unicamente com testes é 97,65%, abaixo da meta de 98%. Considerando somente os 208 mutantes não equivalentes, a suíte detecta 100%. O *StrykerJS* permite excluir mutantes do cálculo com o comentário *// Stryker disable*, mas este deixaria de gerar mutantes naquelas linhas, e uma alteração futura que tornasse essas condições relevantes passaria sem ser avaliada.

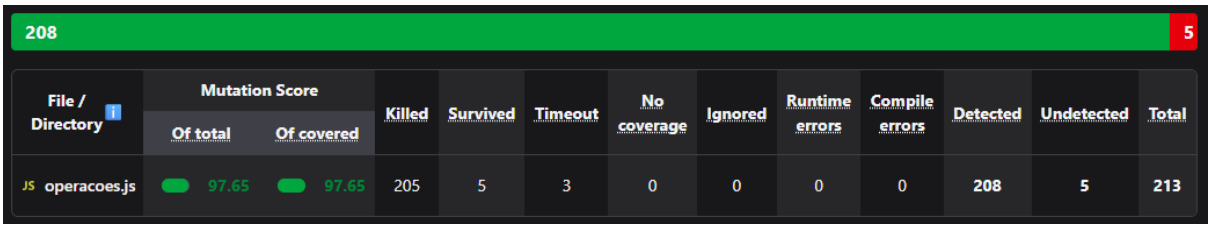


imagem_5 - Relatório StrykerJS

5. Conclusão

A cobertura de código indica as linhas que foram executadas, sem ter o resultado verificado. A suíte original analisada ilustra esta afirmação, tendo quase 99% de linhas cobertas, mas deixando passar mais de um quarto dos defeitos introduzidos. O teste de mutação expôs o padrão do caminho feliz das falhas, sendo entradas de valores felizes sem considerar valores de fronteira e ramos de erro sem verificação.

Também foi observado que mutantes equivalentes impedem a pontuação de cobertura de chegar a 100% e exigem análise manual, reforçando a necessidade de interpretação sobre o teste escrito.